Problem Statement

When a drug causes a bad reaction, patients and doctors can report it to the FDA Adverse Event Reporting System (FAERS). The FDA makes this data public and it contains millions of reports covering almost every drug. Although this data is helpful, it's very inaccessible to non-researchers. The interface is clunky, the data files are large and unclean, and there's no easy way to just look up a drug and see what people have actually been reporting.

This project builds a streamlined web application that lets you search for a drug by name and see the side effects being reported, how often they appear, which ones are trending over time, and how the reports cluster by severity and body system. The application is backed by a SQL database built from the raw FAERS data, with a linear regression model to show whether adverse event reporting for a given drug is increasing or decreasing.

These concepts map directly to the topics discussed in CS 210. The data pipeline involves acquiring data from a public API and bulk CSV files, cleaning it with Pandas, storing it across a relational PostgreSQL database and a NoSQL MongoDB store, applying clustering and regression, and visualizing the results. The Streamlit frontend ties it all together.

Novelty and Importance

Adverse event data is crucial because it shows how drug safety gets tracked after a drug is approved. But tools like FAERS are built for pharmaceutical researchers and regulators, not for patients or students. The OpenFDA API makes the data accessible in theory, but actually doing something useful with it requires cleaning and restructuring that most people can't do.

Harpaz et al. (2012) published a paper in the Journal of the American Medical Informatics Association on using data mining techniques on FAERS to detect drug safety signals, which is essentially what this project is doing in a more accessible form. Additionally, Banda et al. (2016) in Scientific Data notes that FAERS files are inconsistently formatted across years and full of duplicate reports, which is a core part of what this project is addressing in the data cleaning phase. Sloane et al. (2015) in Pharmacoepidemiology and Drug Safety also looked at how patients interpret and search for adverse event information online, and found that most existing tools were too technical for general use.

This is different from existing work because it is the combination of a cleaned, queryable database with a frontend that a non-technical person could use. The tools built on FAERS are either research scripts without any interface, or dashboards that don't expose the underlying data. This project bridges that gap.

Data Sources

FDA FAERS Quarterly Data Files: The FDA publishes quarterly ZIP files containing structured CSVs for each report at https://fis.fda.gov/extensions/FPD-QDE-FAERS/FPD-QDE-FAERS.html. Each quarter includes separate files for demographic info, drug info, reaction terms, and outcomes. I'll use several years of quarterly data so I have enough of a time series for the regression component. The files are publicly available and don't require authentication.

OpenFDA API: The FDA also offers a REST API with the endpoint https://api.fda.gov/drug/event.json that returns adverse event reports in JSON format. This will be used for the live search feature in the application. When a user searches for a drug, the app queries the API to supplement what's already in the local database. Documentation is available at https://open.fda.gov/apis/drug/event/.

Data Storage and Schema Design

Raw JSON responses from the OpenFDA API will go into MongoDB first. The API responses are nested and inconsistently structured, so MongoDB is a better fit before the data gets cleaned and flattened.

The cleaned and structured data will be stored in PostgreSQL with the following schema:

- drugs(drug_id, generic_name, brand_name, manufacturer)
- reports(report_id, drug_id, report_date, age, sex, country)
- reactions(reaction_id, report_id, reaction_term, body_system, severity)
- outcomes(outcome_id, report_id, outcome_type)

The main joins are reports with reactions on report_id and drug_id. I'll index on drug_id and report_date since most queries will be filtering on those. Duplicate reports are a known issue in FAERS, so I'll implement deduplication logic before loading based on a combination of report date, drug name, and reaction term.

Methods and Implementation Plan

ETL Pipeline: The data flows through the pipeline in three stages. First, the raw FAERS quarterly CSV files get downloaded and loaded into Pandas DataFrames for cleaning and transformation. At the same time, live API responses from OpenFDA come in as nested JSON and get stored in MongoDB as-is. Second, both sources go through the cleaning and feature engineering steps described below. Third, the cleaned, structured data gets loaded into PostgreSQL using batch inserts, where it becomes the backend for all the Streamlit queries.

Data Cleaning: FAERS data is messy because drug names are entered in freeform. The same event also may get reported by both the patient and their doctor. I'll handle name normalization using Pandas string operations and regex. and remove duplicates based on a composite key of report date, drug name, and reaction term.

Feature Engineering: For each drug I'll create new features like total report count per quarter, reaction frequency by body system, and a severity score based on outcome type (hospitalization, death, etc.). These features are what the regression model trains on and what gets displayed in the frontend.

Clustering: I'll use K-means clustering on reaction terms to group side effects by body system and co-occurrence. Certain reactions tend to appear together and clustering shows those patterns in a more informative way.

Predictive Model: I'll train a linear regression model to predict quarterly report volume for a given drug based on historical reporting trends. This will flag drugs where reporting is trending upward.

Streamlit Frontend: The application will have three main views. The search view lets users type a drug name and see the top reported reactions, severity breakdown, and demographic info. The trend view shows a time series chart of quarterly report volume with the regression model's forecast overlaid. The cluster view shows a scatter plot of reaction clusters for the selected drug, colored by the body system. All views pull from the PostgreSQL backend.

Evaluation

For the regression model, I'll compare predicted vs. actual quarterly report counts visually using line plots and overlay the model's predictions against the real values for a held-out set of quarters. I'll also use a train/test split (80% train, 20% test) to make sure the model isn't just memorizing the training data.

For the clustering, I'll evaluate by inspecting whether the groups make logical sense by body system. For example, checking that cardiovascular reactions cluster together and aren't mixed in with gastrointestinal ones.

As an overall check, I'll look up the top reported reactions for a few well-known drugs like ibuprofen or metformin and compare them against what's listed in their FDA drug labels. If the app shows reactions that are already documented, the pipeline is working correctly.

References

Banda, J. M., Evans, L., Vanguri, R. S., Tatonetti, N. P., Ryan, P. B., & Shah, N. H. (2016). A curated and standardized adverse drug event resource to accelerate drug safety research. *Scientific Data, 3*, 160026.

FDA. (2024). *FDA Adverse Event Reporting System (FAERS)*. U.S. Food & Drug Administration. https://www.fda.gov/drugs/surveillance/fdas-adverse-event-reporting-system-faers

Harpaz, R., DuMouchel, W., Shah, N. H., Madigan, D., Ryan, P., & Friedman, C. (2012). Novel data-mining methodologies for adverse drug event discovery and analysis. *Clinical Pharmacology & Therapeutics, 91*(6), 1010–1021.

Sloane, R., Osanlou, O., Lewis, D., Mukherjee, D., Murfin, S., & Pirmohamed, M. (2015). Social media and pharmacovigilance: A review of the opportunities and challenges. *British Journal of Clinical Pharmacology, 80*(4), 910–920.

U.S. Food & Drug Administration. (2024). *OpenFDA drug adverse event API.* https://open.fda.gov/apis/drug/event/